Reviewer Pack — Apolar Catalecticant Defect Bounds DISJ Randomized Communica…

Entry 11bf61185540 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 12:01:43 UTC

Apolar Catalecticant Defect Bounds DISJ Randomized Communication

Entry ID: `11bf61185540` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 12:01:43 UTC

1. Conjecture

Field A (mathematical branch): Apolarity / catalecticant rank in classical secant-variety algebraic geometry (Iarrobino-Kanev; Landsberg; rarely applied to communication complexity, <5 papers) **Field B** (complexity object): Two-party randomized communication complexity of DISJ_n and related sign-matrix functions (Razborov 1992; Sherstov pattern matrix)

Statement:

> For any 2-party Boolean function f: {0,1}^n x {0,1}^n -> {0,1}, define the signed weight-profile 3-tensor S_f in R^{(n+1) x (n+1) x (n+1)} by S_f[i,j,k] := (1/Z_{i,j,k}) * sum_{|a|=i, |b|=j, |a cap b|=k} (-1)^{f(a,b)}, where Z_{i,j,k} = binom(n,i)*binom(i,k)*binom(n-i,j-k) is the orbit size (so S_f is a sign-mean per (i,j,k) cell, with S_f=0 if Z=0). Let kappa(f) := rank over Q of the mode-1 unfolding of S_f after thresholding singular values at 1/(n+1)^2. Conjecture (C): for every f, R_{1/3}^{cc}(f) >= (1/8) * kappa(f) * log(n) - O(log n); in particular kappa(DISJ_n) = Theta(n) (saturating the (n+1) bound) while kappa(EQ_n), kappa(GT_n with Alice<Bob trivialized), and kappa of any f with R^{cc}(f) = O(log n) are O(log n). One counterexample = one function with measurable R^{cc} <= c log(n) but kappa >= 2c log(n) + 4 falsifies (C).

Rationale (proposer's reasoning):

> The S_n x S_n-orbit-averaged tensor S_f is the natural symmetric-group projector seen by any communication protocol whose acceptance probability is exchangeable in the indices (which all known DISJ protocols essentially are, by Razborov's symmetrization). Apolarity says the unfolding rank of S_f counts the apolar Hilbert function of the polarized form sum f(a,b) x^a y^b, an invariant of the underlying secant geometry on the Segre P^n x P^n. The conjecture predicts that this apolar defect is exactly what forces high randomized cost on DISJ while collapsing on permutation-invariant low-communication functions like EQ.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8f25e3b375177cd4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds at each n∈{6,10,16,24,40}, compute κ(f) for f∈{DISJ,EQ,GT,RANDOM}. Conjecture is SUPPORTED iff median κ(EQ_n) and κ(GT_n) ≤ 2·log2(n)+4 for ALL n, AND median κ(DISJ_n)/n ≥ 0.25 for n≥16. FALSIFIED if any seed yields κ(EQ_n) ≥ 2·log2(n)+4 at any n≥10.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `catalecticant rank communication complexity lower bound` - `apolarity tensor decomposition disjointness Razborov` - `symmetric tensor rank sign matrix pattern matrix Sherstov`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def binomial(n, k):
    if k > n:
        return 0
    if k == 0 or k == n:
        return 1
    result = 1
    for i in range(1, k + 1):
        result *= (n - i + 1)
        result //= i
    return result

def sign(x):
    return 1 if x else -1

def S_f(f, n, i, j, k):
    Z = binomial(n, i) * binomial(i, k) * binomial(n - i, j - k)
```

```
    if Z == 0:
        return 0
    count = 0
    for _ in range(min(200, Z)):
        a = [random.randint(0, 1) for _ in range(i)]
        b = [random.randint(0, 1) for _ in range(j)]
        if len(set(a).intersection(b)) == k:
            count += sign(f(tuple(a), tuple(b)))
    return Fraction(count, Z)

def kappa(f, n):
    S = [[S_f(f, n, i, j, k) for k in range(n + 1)] for j in range(n + 1)]
    singular_values = []
    for i in range(n + 1):
        row = [S[i][j] for j in range(n + 1)]
        col = [S[j][i] for j in range(n + 1)]
        max_val = max(abs(row), abs(col))
        if max_val > 0:
            singular_values.extend([max_val, 1 / (n + 1) ** 2])
    singular_values.sort(reverse=True)
    rank = sum(1 for sv in singular_values if sv > 1 / (n + 1) ** 2)
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 10, 16, 24, 40]
    kappa_values = []

    def DISJ_n(x, y):
        return all(x[i] == y[i] for i in range(n))

    def EQ_n(x, y):
        return x == y

    def GT_n(x, y):
        return sum(x) > sum(y)

    functions = [DISJ_n, EQ_n, GT_n]

    for n in n_values:
        kappa_values.extend([kappa(f, n) for f in functions])

    metric_value = max(kappa_values)
    instances_tested = len(kappa_values)
    conjecture_holds = all(kv <= 2 * math.log2(n) + 4 for kv, n in zip(kappa_values[:len(n_values)
                       kappa_values[len(n_values):] >= [0.25 * n for n in n_values[2:]]

    return {
        "metric_name": "kappa_max",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
```

```python
import sys
seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif any(r["kappa_values"][len(n_values):] >= [0.25 * n for n in n_values[2:]] for r in result
    print("RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed=1")
else:
    print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6afff1a0.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6afff1a0.py", line 74, in run_trial
    kappa_values.extend([kappa(f, n) for f in functions])
                         ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6afff1a0.py", line 45, in kappa
    S = [[S_f(f, n, i, j, k) for k in range(n + 1)] for j in range(n + 1)]
                    ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an UnboundLocalError in the kappa function before producing any data, so neither the support nor falsification criteria can be evaluated. | next: Fix the scoping bug in kappa/S_f (pass i explicitly or correct the variable binding) and rerun the pre-registered sweep across n∈{6,10,16,24,40} with 30 seeds.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	229881
2	preregistration	claude_max	opus	0	0	6853
3	novelty	claude_max	opus	0	0	3418
4	novelty	claude_max	opus	0	0	8524
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21478
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16126
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11802
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11815
9	judge	claude_max	opus	0	0	5070

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 314966 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/11bf61185540.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/11bf61185540.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/11bf61185540.tar.gz` (if generated)